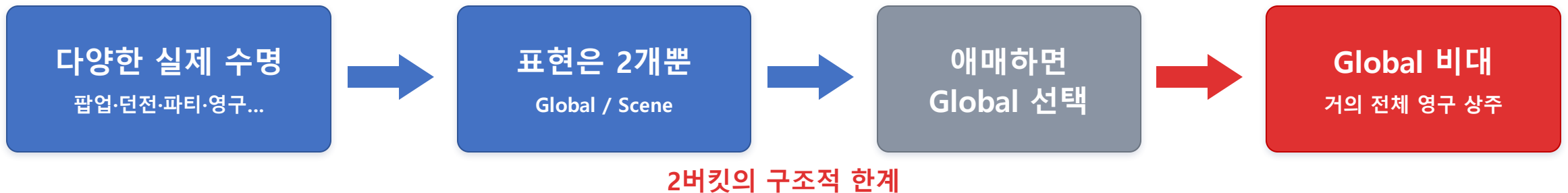


문제상황 (2) - 1

리소스 수명 관리 : 2버킷 구조의 한계와 수명 스코프 도입

🚨 문제 상황

- 리소스의 실제 수명은 5종 이상인데 표현 수단은 **Global / Scene 2개뿐** — 수명이 애매한 리소스가 전부 "**안전한 쪽 = Global**"로 도피



1. 수명 표현력 부족

- 팝업(스택 동안)·파티(씬 넘어 생존)·던전(던전 동안)을 담은 **중간 수명 계층이 없음**

2. Global 도피 관성

- "언제 Release?"가 애매 → 안전한 Global 선택 → **한 번 올리면 게임 종료까지 안 내려옴**

3. 그 결과

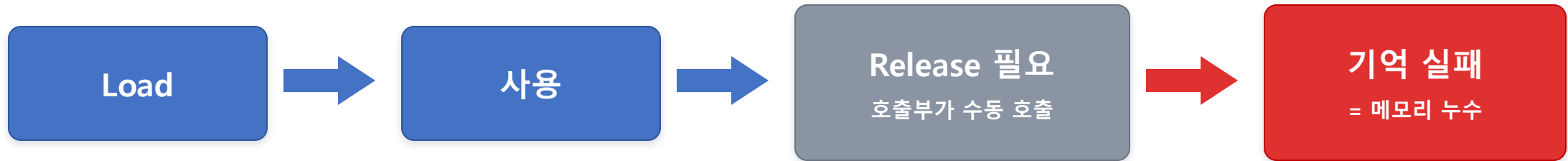
- 진입 직후 핸들 메모리의 **95%가 Global 버킷** (1287MB vs Scene 65MB) — 거의 전체가 항상 메모리 상주

문제상황 (2) - 2

리소스 수명 관리 : 수동 짝맞추기의 함정과 실측 비용

🚨 문제 상황

- Addressables는 **Load ↔ Release 짝맞추기**를 요구(공식 문서) — 호출부가 Release를 '기억'해야 하고, 그 **기억의 실패가 곧 메모리 누수**



수동 짝맞추기의 함정

1. 팝업 세션의 영구 고정

- 팝업 한 세션 열람 = Global에 **+287MB 영구 고정** — 닫아도.싹을 옮겨도 게임 종료까지 안 내려옴

2. 몬스터 HP바 누수 (실제 발견·수정)

- DontDestroyOnLoad 캔버스에 방치돼 자가정리가 안 됨 → Memory Profiler **GameObject +291**

3. 구조적 원인

- 수명 관리가 **개인의 주의력**에 의존 → 실패가 반복 → "기억"이 아니라 **구조로 정산**할 필요

→ 개인의 기억이 아니라 **한 곳에서 수명 단위로 자동 정산**하는 통합 관리가 필요 (→ 문제해결)

🔗 프로젝트 목차로 이동